

第3章 HBase | 开卷速查版

本章一句话：**HBase** 是构建在 **HDFS** 之上的分布式列族存储系统，用来解决 HDFS/MapReduce 不擅长的 **高并发、随机、实时读写** 问题。它适合存储超大规模稀疏表，通过 **RowKey** 快速定位数据。☺

常考题型： HBase vs HDFS、HBase 数据模型、RowKey/列族/时间戳、LSM-tree、物理模型、架构组件、读写流程、Compaction、Feed 流表设计。

翻页关键词： HBase动机 → HBase特点 → Key-Value模型 → Map of Maps → B+Tree vs LSM → Region/Store/MemStore/HFile → 架构 → Region定位 → 写流程 → 读流程 → Compaction → Feed流设计。

1. 为什么需要 HBase

HDFS + MapReduce 适合大文件顺序读写和离线批处理，但不适合高并发随机读写。比如社交网络、消息系统、搜索索引、监控时序数据，都需要根据某个 key 快速读写少量数据。PPT里明确说：GFS 和 MapReduce 没有满足“可以高并发、保障一致性，并且支持随机读写数据”的需求，BigTable 和 HBase 就是为了解决这个需求。

HBase 的定位： 它不是替代 HDFS，而是建立在 HDFS 上，用 HDFS 存底层文件，同时对外提供类似数据库的 **Put / Get / Scan** 操作。

考试写法： HBase 的出现是为了弥补 HDFS 和 MapReduce 的不足。HDFS 适合大文件顺序读写，MapReduce 适合离线批处理，但它们不适合高并发、低延迟、随机读写。HBase 构建在 HDFS 之上，提供面向 RowKey 的随机实时读写能力，适合海量结构化或半结构化数据存储。

2. HBase vs HDFS

HDFS 是文件系统。 它存大文件，适合顺序读写和批处理，不支持随机更新，也不适合频繁插入删除。

HBase 是数据库/存储系统。 它底层文件存在 HDFS 上，但对外表现为一张超大表，支持按 **RowKey** 的随机读写。

对比点	HDFS	HBase
本质	分布式文件系统	分布式列族数据库
数据单位	文件 / Block	表 / Row / Column Family / Cell
访问方式	顺序读写大文件	Put、Get、Scan
适合场景	离线批处理、大文件	随机实时读写、海量稀疏表
更新能力	不支持随机修改	支持按 RowKey 写入/更新
底层关系	被 HBase 使用	建在 HDFS 上

PPT里也直接对比：HDFS 适合批处理，不支持数据随机查找，不适合随机插入/删除，也不支持数据更新。

考试写法： HDFS 是底层分布式文件系统，适合大文件顺序读写和批处理；HBase 是构建在 HDFS 之上的分布式列族存储系统，支持基于 RowKey 的随机实时读写。HBase 通过 HDFS 获得可靠存储能力，同时在其上提供更接近数据库的访问接口。

3. HBase vs DBMS / MySQL

PPT中对 MySQL 和 HBase 的比较口径很直接：MySQL 是行存，HBase 是列族；MySQL 更偏 **scale-up**，HBase 更偏 **scale-out**；MySQL 支持 SQL 和 join，HBase 主要通过 RowKey 查询，不支持传统 SQL 和 join。

对比点	MySQL / DBMS	HBase
模型	关系模型、二维表	列族模型、稀疏大表
查询	SQL	RowKey / Scan
Join	支持	不支持或不擅长
事务	强事务能力	单行强一致为主
扩展	scale-up 为主	scale-out 为主
适合	复杂查询、事务系统	海量数据、简单高并发读写

考试写法：传统 DBMS 适合结构化数据、复杂 SQL、join 和事务处理；HBase 牺牲复杂 SQL 和 join 能力，换取水平扩展、高并发随机读写和海量稀疏数据存储能力。

4. HBase 的特点

HBase 的特点可以背五个词：**大、无模式、列族、稀疏、多版本**。

大：一个表可以有数十亿行、上百万列，适合海量数据。

无模式：每行可以有不同列，列可以动态增加。建表时主要固定的是 **Column Family 列族**，具体列不一定提前定义。

面向列族：HBase 的物理存储和权限控制按 **Column Family** 组织。列族建表时确定，列族下的列可以动态扩展。

稀疏：空列不占空间，所以一张表可以有很多列，但每行只用其中少数列。

多版本：一个 Cell 可以按 **Timestamp** 保存多个版本，默认时间戳通常是写入时间。

PPT明确列出 HBase 的特点：**大、无模式、面向列、稀疏、数据多版本**。

考试写法：HBase 适合管理海量稀疏数据。它支持数十亿行和上百万列，采用无模式设计，不同行可以拥有不同列；数据按列族组织，空列不占空间；同一个单元格还可以保存多个时间戳版本。

5. HBase 为什么能管理大规模数据

HBase 能管理大规模数据，靠的不是单机能力，而是 **Region 分片 + RegionServer 分布式存储 + HDFS 容错 + ZooKeeper 协调**。

核心逻辑：

—张大表
-> 按 RowKey 范围切成多个 Region
-> 不同 Region 分配到不同 RegionServer
-> RegionServer 负责读写
-> 底层 StoreFile/HFile 存在 HDFS
-> Master 负责分配、负载均衡、故障恢复

所以，HBase 的扩展方式是：表大了以后，**Region** 可以继续分裂，新 Region 可以被分配到不同机器上，从而把数据量和访问压力分散出去。

考试写法： HBase 通过将表按 RowKey 范围切分为多个 Region，并把不同 Region 分配给不同 RegionServer，实现数据和负载的分布式管理。底层数据文件存储在 HDFS 上获得可靠性，Master 和 ZooKeeper 负责协调、分配和故障恢复，因此可以扩展到大规模集群。

6. HBase 为什么也能用于批处理

严格说，HBase 不是批处理计算框架，它是存储系统。但它可以很好地和 **MapReduce / Spark** 配合，用来做批量扫描和索引构建。

PPT里举了互联网搜索例子：爬虫抓取新页面后，每页一行存入 HBase；MapReduce 作业运行在整张 HBase 表上生成索引，为搜索应用做准备。

所以可以这样理解：

实时访问： 用户按 RowKey 查询单个文档。**批处理：** MapReduce/Spark 扫描整张或一部分 HBase 表，批量生成索引、统计结果。

考试写法： HBase 本身是存储系统，但适合与 MapReduce 等批处理框架结合。HBase 可以存储海量结构化数据，批处理框架可以对整张表或一批 Region 进行扫描和计算，例如搜索引擎中用 HBase 存网页，再用 MapReduce 批量生成索引。

7. Key-Value 模型

HBase 表面上像一张表，但底层更接近 **Key-Value 存储**。

一个 Cell 的完整定位方式是：

(RowKey, ColumnFamily:Qualifier, Timestamp) -> Value

也就是：

哪一行 + 哪个列族 + 哪个列 + 哪个版本 -> 具体值

例子：

```
RowKey = user_001
Column = info:name
Timestamp = 1001
Value = Alice
```

RowKey 是最重要的设计点。HBase 会按 RowKey 字典序排序，并按 RowKey 范围切 Region。所以 RowKey 决定查询效率、负载均衡和 Scan 是否方便。

8. BigTable & HBase 基本概念

Table：表，逻辑上的大表。

RowKey：行键，唯一标识一行，按字典序排序。HBase 设计题第一优先级就是设计 RowKey。

Column Family：列族，建表时确定，是物理存储单位。不同列族通常分开存。

Qualifier：列名，列族下面的具体列，可以动态增加。

Timestamp：时间戳，用于多版本。

Cell：一个具体单元格，由 RowKey、Column Family、Qualifier、Timestamp 唯一确定，值是 byte array。

Region：表按 RowKey 范围切出来的分片，是 HBase 分布式存储和负载均衡的基本单位。

9. Map of Maps

HBase 的逻辑数据模型可以写成 **Map of Maps**：

```
Map<
  RowKey,
  Map<
    ColumnFamily,
    Map<
      Qualifier,
      Map<Timestamp, Value>
    >
  >
>
```

更直白地写：

```
行键 -> 列族 -> 列 -> 时间戳版本 -> 值
```

例子：

```
user_001
  info
    name
      t3 -> "Alice"
    age
      t3 -> "20"
  post
    p100
      t5 -> "hello"
```

考试写法： HBase 的数据模型可以理解为多层 Map：RowKey 映射到列族，列族映射到具体列，具体列再映射到多个时间戳版本，最终得到 value。因此 HBase 可以支持稀疏列和多版本数据。

10. B+Tree vs LSM-tree

这个是理解 HBase 写快的核心。

B+Tree： 传统数据库常用。数据按树结构组织，读很快，范围查询也方便。但随机写入时可能要不断修改磁盘上的树节点，随机 I/O 多。

LSM-tree：Log-Structured Merge Tree。 核心思想是：先把随机写变成内存写和顺序写。数据先写日志，再写内存结构，内存满了以后顺序刷到磁盘文件。后续通过合并文件减少读放大。

对比：

对比点	B+Tree	LSM-tree
写入方式	直接修改树节点	先写 WAL + MemStore，再顺序刷盘
写性能	随机写较重	写入吞吐高
读性能	通常较稳定	可能查多个文件
适合场景	传统数据库	高写入吞吐、分布式存储

HBase 使用 LSM 思想：

```
Put
-> 写 WAL
-> 写 MemStore
-> MemStore 满后 flush 成 StoreFile/HFile
-> 多个 StoreFile 后 compaction 合并
```

PPT里写操作相关页面强调了 **Write-Ahead-Log (WAL)**，HFile 又按 block 组织，默认 block size 为 64KB。

考试写法： HBase 采用 LSM-tree 思想，将随机写转化为顺序写。写入时先追加 WAL 保证故障恢复，再写入内存中的 MemStore；MemStore 达到阈值后顺序 flush 成 HFile。这样提高写入吞吐，但读操作可能需要同时

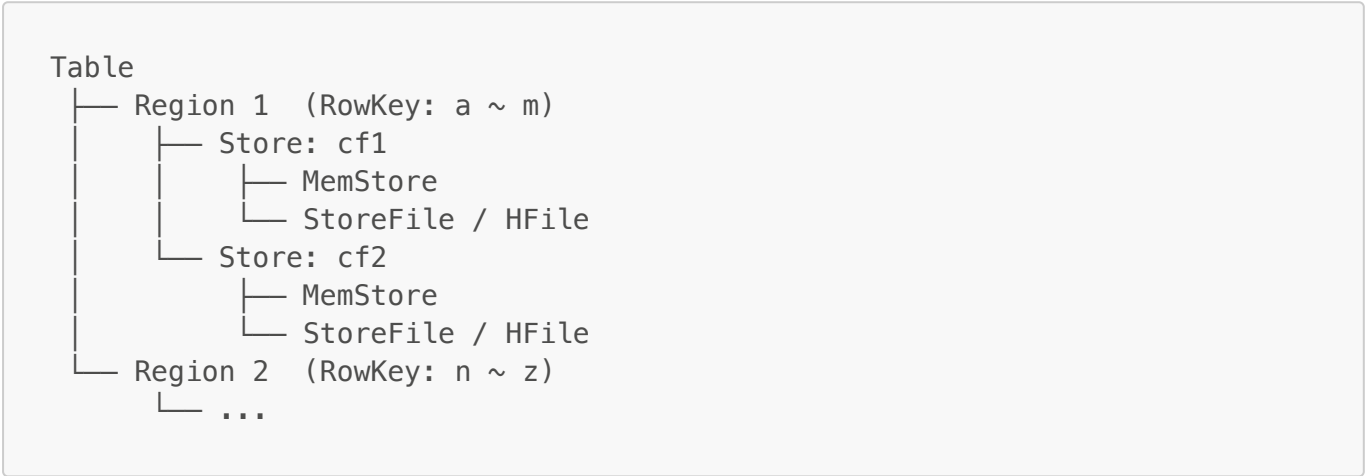
查 MemStore 和多个 HFile，因此需要 Compaction 合并文件。

11. HBase 逻辑模型 vs 物理模型

逻辑模型： 用户看到的是一张表，有 RowKey、列族、列、时间戳和值。

物理模型： HBase 真正存储时，会按 RowKey 范围把表切成多个 **Region**，每个 Region 分配给某个 **RegionServer**。Region 内部按列族分成多个 **Store**，每个 Store 由 **MemStore + StoreFile/HFile** 组成。

结构可以画成：



记忆口诀：

Table 按 RowKey 切 Region
Region 按 Column Family 切 Store
Store = MemStore + HFile
HFile 存在 HDFS 上

12. HBase 物理存储

Region 是水平切分：按 RowKey 范围分片。

Store 是列族切分：一个 Column Family 对应一个 Store。

MemStore 是内存写缓存。新写入数据先进入 MemStore。

StoreFile / HFile 是磁盘文件。MemStore 满了以后 flush 到 HDFS 上形成 HFile。

HFile 是 HBase 的底层文件格式，按 block 组织。PPT给出 HFile block 默认大小是 **64KB**。

注意 HBase block 和 HDFS block 不一样：

对比点	HBase Block	HDFS Block
默认大小	PPT中 HFile block 默认 64KB	PPT中 HDFS block 默认 64MB
层级	HBase/HFile 内部读写缓存单位	HDFS 底层文件存储单位

对比点	HBase Block	HDFS Block
作用	方便随机读、缓存、索引	适合大文件顺序读写
粒度	小	大

HDFS block 是文件系统底层块，PPT里 HDFS 默认 block 为 64MB，每个 block 有多个副本。

考试写法： HBase 的 HFile block 是 HBase 内部读写和缓存单位，默认约 64KB；HDFS block 是底层分布式文件系统的存储单位，默认 64MB。HBase 的 HFile 最终作为普通文件存放在 HDFS 上，因此一个 HFile 会被 HDFS 切成多个 HDFS block。

13. Key-Value Format

HBase 底层不是简单只存 value，而是把每个 Cell 存成带完整 key 信息的 **KeyValue**。

可以写成：

```
KeyValue = RowKey + ColumnFamily + Qualifier + Timestamp + Type + Value
```

其中：

RowKey：哪一行。 **ColumnFamily + Qualifier**：哪一列。 **Timestamp**：哪个版本。 **Type**：Put / Delete 等操作类型。 **Value**：真正的值。

这样做的好处是：HFile 中的数据按 key 排序，HBase 可以通过 RowKey、列和版本快速定位，也能处理多版本和删除标记。

14. HBase 架构组件

HBase 架构主要有四个角色：**Client**、**HMaster**、**RegionServer**、**ZooKeeper**。PPT架构部分对应 HBase 架构、读写和容错。

Client：发起 Put/Get/Scan。Client 会缓存 Region 位置信息，避免每次都重新定位。

HMaster / Master：管理者。负责 Region 分配、负载均衡、RegionServer 故障处理、表结构变更等。注意：普通读写一般不经过 Master。

RegionServer：真正处理读写请求。它管理多个 Region，每个 Region 负责一段 RowKey 范围。

ZooKeeper：协调服务。负责 Master 选举、保存关键元数据入口、监控 RegionServer 状态等。

图可以画成：

```
Client
|
| 定位 Region 后直接读写
v
RegionServer 1 ---- HDFS
```

```
RegionServer 2 ---- HDFS
RegionServer 3 ---- HDFS

HMaster: 分配 Region / 负载均衡 / 故障恢复
ZooKeeper: 协调 / 选主 / 保存入口 / 心跳状态
```

考试写法： HBase 由 Client、Master、RegionServer 和 ZooKeeper 组成。Client 发起读写并缓存 Region 位置；Master 负责 Region 分配、负载均衡和故障处理；RegionServer 负责管理 Region 并处理实际读写；ZooKeeper 负责协调、选主和维护关键状态信息。

15. Region 定位

HBase 读写之前，Client 必须先知道：**目标 RowKey 属于哪个 Region，这个 Region 在哪个 RegionServer 上。**

按 PPT 常见旧版口径可以写：

```
ZooKeeper
-> -ROOT-
-> .META.
-> 用户表 Region
-> 对应 RegionServer
```

更直观地说：

1. Client 先通过 ZooKeeper 找到元数据表入口。
2. 再查元数据表，找到目标 RowKey 所在 Region。
3. 得到该 Region 所在 RegionServer 地址。
4. Client 直接访问 RegionServer。
5. Client 缓存该位置，后续访问更快。

新版本 HBase 中，**-ROOT-** 已经简化，通常是 ZooKeeper 找到 **hbase:meta** 的位置，再从 **hbase:meta** 定位用户表 Region。考试如果按 PPT，写 **ZooKeeper → -ROOT- → .META. → 用户表 Region** 更保险。

考试写法： HBase 的读写首先需要 Region 定位。Client 通过 ZooKeeper 找到元数据表入口，再查询元数据表得到目标 RowKey 所在 Region 以及对应 RegionServer 地址，之后直接访问该 RegionServer，并将位置信息缓存在本地以减少后续定位开销。

16. HBase 写操作流程

写流程是高频题，按这个顺序背：

1. Client 根据 RowKey 做 Region 定位。
2. 找到对应 RegionServer。
3. RegionServer 先写 WAL。

4. 再把数据写入 MemStore。
5. 写入成功后返回 ack。
6. MemStore 达到阈值后 flush 成 StoreFile/HFile。
7. StoreFile 多了以后执行 Compaction。

为什么先写 WAL? 因为 MemStore 在内存里，机器挂了会丢。WAL 是预写日志，先把操作追加到日志中，故障后可以用 WAL 恢复。

为什么写快? 因为写 WAL 是追加写，MemStore 是内存写，flush 成 HFile 也是顺序写。它避免了大量随机磁盘修改。

考试写法: HBase 写入时，Client 先根据 RowKey 定位 Region 和 RegionServer。RegionServer 收到写请求后，先将操作追加到 WAL 以保证故障恢复，再写入内存中的 MemStore。MemStore 达到阈值后会 flush 到 HDFS 形成 HFile，多个 HFile 后通过 Compaction 合并。该机制将随机写转为顺序写，因此写入吞吐较高。

17. HBase 读操作流程

PPT里读操作可以压成两步：**Region定位**，然后从 **MemStore** 和 **StoreFile** 里找数据。

完整流程：

1. Client 根据 RowKey 定位 RegionServer。
2. 到对应 Region 的 Store 中查找。
3. 先查 MemStore，因为最新写入可能还没 flush。
4. 再查 BlockCache，如果缓存中有 HFile block，直接读缓存。
5. 再查 HFile / StoreFile。
6. 合并 MemStore 和多个 HFile 中的结果。
7. 根据 timestamp、删除标记、版本数返回最终结果。

为什么读可能比写复杂? 因为数据可能散落在 MemStore 和多个 HFile 中。HBase 需要把多个位置的结果合并，挑出最新版本或符合条件的版本。

考试写法: HBase 读取时，Client 先定位目标 RowKey 所在 RegionServer，然后在对应 Region 的 Store 中查找数据。读操作需要同时检查 MemStore、缓存和多个 StoreFile/HFile，并根据时间戳、版本和删除标记合并结果。因此 HBase 写入较快，但读取可能受到 StoreFile 数量影响。

18. Compaction 是什么

Compaction 是 HBase 后台合并 StoreFile/HFile 的过程。

为什么需要? MemStore 每次 flush 都会生成新的 HFile。时间久了，一个 Store 下会有很多 HFile。读数据时可能要查很多文件，读性能下降。所以需要合并。

两类：

Minor Compaction: 合并一部分小 HFile，减少文件数量。

Major Compaction: 合并一个 Store 下所有 HFile，同时清理删除标记和过期版本。效果更彻底，但开销更大。

考试写法: Compaction 是 HBase 将多个 StoreFile/HFile 合并的后台过程。它可以减少文件数量，降低读操作需要检查的文件数，并在 Major Compaction 中清理删除标记和过期版本。Minor Compaction 合并部分文件，Major Compaction 合并全部文件，后者开销更大。

19. HBase 容错机制

HBase 的容错主要靠 **HDFS 副本、WAL、ZooKeeper、Master 重分配 Region**。

RegionServer 故障: ZooKeeper 发现 RegionServer 心跳异常，Master 会把它负责的 Region 重新分配给其他 RegionServer。未 flush 的内存数据可以通过 WAL 回放恢复。

Master 故障: ZooKeeper 可以重新选主。Master 不直接处理普通读写，所以短时间 Master 挂掉通常不会立刻影响已有 Region 的读写，但会影响 Region 分配、负载均衡和故障恢复。

数据文件可靠性: HFile 存在 HDFS 上，由 HDFS 多副本保证底层可靠性。

考试写法: HBase 的容错依赖 HDFS、WAL 和 ZooKeeper。HFile 存储在 HDFS 上，通过多副本保证可靠性；RegionServer 故障时，Master 会将其 Region 重新分配到其他 RegionServer，并通过 WAL 回放恢复未刷盘数据；Master 故障可通过 ZooKeeper 重新选主。

20. 读写总结：一句话版

写: 找 Region → 写 WAL → 写 MemStore → flush 成 HFile → compaction。 **读:** 找 Region → 查 MemStore → 查缓存/HFile → 合并版本和删除标记。 **写快原因:** 内存写 + 顺序写。 **读慢原因:** 可能查多个 HFile，需要合并。 **Compaction 作用:** 减少 HFile 数量，提升读性能，清理旧版本/删除标记。

21. Feed 流思考题：场景描述

Feed 流就是社交软件的时间线。比如用户打开首页，要看到自己关注的人最近发的帖子。

这个题的核心是：**读扩散 vs 写扩散**。

假设 A 发了一条帖子，A 有很多粉丝。系统可以选择：

发帖时推给粉丝，读的时候很快，但写压力大。 **读的时候去拉关注者帖子**，写的时候轻松，但读压力大。

22. Feed 流方法 1：读扩散 / Pull / On Read

做法: 用户打开首页时，系统查询他关注的所有人的发件箱，再按时间合并排序。

用户 U 关注 A、B、C

读首页时：

查 A 的 outbox

查 B 的 outbox

查 C 的 outbox
合并排序

优点：发帖时只写自己的发件箱，写入压力小。**缺点：**读首页很重，关注人数多时要查很多人。**适合：**关注人数少、读频率低的场景。

考试写法：读扩散是在读取 Feed 时查询用户关注对象的发件箱并合并排序。它写入简单，但读操作会随着关注人数增加而变重，适合关注关系较少或读频率较低的用户。

23. Feed 流方法 2：写扩散 / Push / On Write

做法：用户发帖时，系统把这条帖子的索引写入所有粉丝的收件箱。

```
A 发帖 post_1
粉丝有 U1, U2, U3
写入：
  U1 inbox <- post_1
  U2 inbox <- post_1
  U3 inbox <- post_1
```

优点：读首页快，只需要读自己的 inbox。**缺点：**大 V 发帖会产生巨量写入，写放大严重。**适合：**普通用户、粉丝数量不大的账号。

PPT前面 Friendster 例子也展示了 on write 会把写入放大到好友数倍，例如 150 个好友时写入放大 150 倍。

考试写法：写扩散是在用户发帖时将帖子索引写入所有粉丝的收件箱。它使读 Feed 很快，但会造成写放大，尤其大 V 用户发帖时会产生大量随机写入，因此不适合粉丝数极大的用户。

24. Feed 流方法 3：混合推拉 / Hybrid Fanout

做法：普通用户用写扩散，大 V 用读扩散或只推给活跃粉丝。

```
普通用户发帖：
  推到所有粉丝 inbox

大 V 发帖：
  写自己的 outbox
  可选：只推给活跃粉丝 inbox

用户读首页：
  读自己的 inbox
  再拉取关注的大 V outbox
  合并排序
```

优点：平衡读写压力。**缺点：**系统设计更复杂，需要区分普通用户、大 V、活跃粉丝。

考试写法：混合推拉结合了读扩散和写扩散。普通用户发帖时推送到粉丝收件箱，大 V 发帖时只写自己的发件箱或只推送给活跃粉丝。用户读取 Feed 时读取收件箱，并补充拉取关注大 V 的发件箱内容。这样可以平衡大 V 写放大和普通用户读延迟。

25. Feed 流 HBase 表结构设计

PPT里表结构设计目标是支持 Feed 流、平衡写扩散与读扩散、提升活跃用户读取性能；核心思想是 **内容与索引分离、推拉结合、多表设计**，核心表包括 `user_profile`、`followee_list` / `follower_list`、`post_content`、`user_inbox`、`user_outbox`。

和人相关的表

表名	RowKey	列族/列	用途
<code>user_profile</code>	<code>user_id</code>	<code>info:is_vip</code> 、 <code>info:active_level</code>	存用户类型，如是否大 V、是否活跃
<code>followee_list</code>	<code>user_id</code>	<code>normal:followee_id</code> 、 <code>vip:followee_id</code>	某用户关注了谁
<code>follower_list</code>	<code>user_id</code>	<code>fans:follower_id</code>	某用户有哪些粉丝
<code>active_follower_list</code>	<code>user_id</code>	<code>fans:follower_id</code>	大 V 的活跃粉丝列表

这里的访问模式：

```
查 user_id 的关注列表 -> Get/Scan followee_list
查 user_id 的粉丝列表 -> Get/Scan follower_list
判断是否大 V -> Get user_profile
```

和帖子相关的表

表名	RowKey	列族/列	用途
<code>post_content</code>	<code>post_id</code>	<code>meta:user_id</code> 、 <code>meta:time</code> 、 <code>cnt:text</code>	存帖子正文
<code>user_outbox</code>	<code>user_id</code>	<code>posts:reverse_ts#post_id</code>	用户自己发过什么
<code>user_inbox</code>	<code>user_id</code>	<code>posts:reverse_ts#post_id</code>	用户首页时间线索引

为什么用 `reverse_ts#post_id`？因为 HBase 按列名或 RowKey 字典序排列。用倒序时间戳可以让最新帖子排在前面，方便首页读取最近 N 条。

内容与索引分离：`user_inbox` / `user_outbox` 只存 `post_id` 这种索引，不重复存正文。真正正文存在 `post_content`。这样可以减少写扩散时的数据量。

考试写法：Feed 流设计中，应采用内容与索引分离。帖子正文存入 `post_content`，用户发件箱 `user_outbox` 和收件箱 `user_inbox` 只保存按倒序时间组织的 `post_id` 索引。普通用户发帖时将索引推入

粉丝 inbox，大 V 发帖时写 outbox 并按需推给活跃粉丝。读取时先读用户 inbox，再拉取关注大 V 的 outbox，最后根据 post_id 批量读取正文。

本章高频简答模板

1. HBase 解决了什么问题？ HBase 解决 HDFS 和 MapReduce 不适合高并发随机读写的问题。HDFS 适合大文件顺序访问，MapReduce 适合离线批处理，而 HBase 构建在 HDFS 上，提供基于 RowKey 的随机实时读写能力，适合海量稀疏表和简单高并发访问。

2. HBase 的数据模型是什么？ HBase 的 Cell 由 RowKey、ColumnFamily、Qualifier 和 Timestamp 唯一确定，可表示为 `(RowKey, ColumnFamily:Qualifier, Timestamp) -> Value`。整体上可以理解为多层 Map：RowKey 到列族，列族到列，列到时间戳版本，最终得到 value。

3. HBase 为什么写入快？ HBase 采用 LSM-tree 思想，写入时先追加 WAL，再写入内存 MemStore，MemStore 满后顺序 flush 成 HFile。它避免直接随机修改磁盘文件，将随机写转化为顺序写，因此写入吞吐高。

4. Compaction 的作用是什么？ Compaction 用于合并多个 StoreFile/HFile，减少读操作需要访问的文件数量，并清理过期版本和删除标记。Minor Compaction 合并部分小文件，Major Compaction 合并全部文件，后者开销更大。

5. Region 是什么？ Region 是 HBase 表按 RowKey 范围切分得到的分片，是分布式存储和负载均衡的基本单位。不同 Region 可以分配给不同 RegionServer，从而实现水平扩展。

6. HBase 读写流程分别是什么？ 写流程是：Region 定位 → 写 WAL → 写 MemStore → flush 成 HFile → compaction。读流程是：Region 定位 → 查 MemStore → 查缓存和 HFile → 合并多个版本和删除标记 → 返回结果。

7. HBase 为什么不适合复杂 SQL 和 Join？ HBase 主要按 RowKey 组织和访问数据，适合简单的 Get、Put 和 Scan，不提供传统关系数据库的 SQL 优化器、复杂 join 和跨表事务能力。因此 HBase 表设计要围绕访问模式提前设计 RowKey 和列族。

本章易混点

HBase 是列族存储，不是普通列式数据库。 它按 Column Family 物理组织，列族内的列可以动态增加。

HBase 建在 HDFS 上，但访问方式不同。 HDFS 读写文件；HBase 读写表中的行和 Cell。

RowKey 是最重要的设计点。 RowKey 决定数据排序、Region 切分、查询效率和热点风险。

HFile block 和 HDFS block 不同。 HFile block 是 HBase 文件内部缓存/读取单位；HDFS block 是底层文件系统存储单位。

写快不代表读一定快。 HBase 写入通过 WAL + MemStore + 顺序 flush 提高吞吐；读取可能要查多个 HFile，所以需要 BlockCache 和 Compaction。

删除不一定立刻物理删除。 HBase 常通过删除标记表示删除，真正清理通常在 Major Compaction 时完成。

本章看到题目怎么写

看到“**HBase vs HDFS**”：写文件系统 vs 随机读写数据库；HDFS 顺序批处理，HBase RowKey 实时读写。

看到“**HBase 数据模型**”：写 (RowKey, CF:Qualifier, Timestamp) -> Value，再写 Map of Maps。

看到“**为什么 HBase 写快**”：写 LSM-tree、WAL、MemStore、顺序 flush、Compaction。

看到“**Region / 架构**”：写表按 RowKey 切 Region，RegionServer 管 Region，Master 负载均衡，ZooKeeper 协调。

看到“**读写流程**”：写 Region 定位，再分别写 WAL/MemStore/HFile 或 MemStore/HFile 合并。

看到“**Feed 流设计**”：写读扩散、写扩散、混合推拉；表设计写 user_profile、关注/粉丝表、post_content、user_inbox、user_outbox。